

PROJECT REPORT

ON

“CampusSphere: A Unified Platform for College Events, Club Treasuries, Verified Attendance and AICTE Activity Points”

Submitted in partial fulfillment of the requirement for the award of Degree of Bachelor of
Technology in Computer Science & Engineering

Submitted to:



Rajasthan Technical University, Kota (Raj.)

Submitted By:

Govind Jangid (Team Leader) (24EARCA051)

Garvita Jain (_____)

Ayushi Garg (_____)

Ankit Yadav (_____)

Anshika Pathak (_____)

Under the guidance of

PROJECT GUIDE

Er. Ram Babu Buri

Department of Computer Science & Engineering



Session: 2026-27

Department of Computer Science & Engineering

**ARYA COLLEGE OF ENGINEERING & I.T.
SP-42, RIICO INDUSTRIAL AREA, KUKAS, JAIPUR**

CERTIFICATE

This is to certify that the project report entitled “**CampusSphere: A Unified Platform for College Events, Club Treasuries, Verified Attendance and AICTE Activity Points**”, submitted by **Govind Jangid (24EARCA051), Garvita Jain, Ayushi Garg, Ankit Yadav and Anshika Pathak** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science & Engineering** of **Rajasthan Technical University, Kota**, is a record of the bonafide work carried out by the group at **Arya College of Engineering & I.T., Jaipur** during the session 2026-27, under my guidance and supervision.

To the best of my knowledge, the work embodied in this report has not been submitted to any other university or institute for the award of any degree or diploma.

Date: _____ Place: Jaipur

Er. Ram Babu Buri
Project Guide
Dept. of CSE, ACEIT, Jaipur

Dr. Akhil Pandey
Head of Department
Dept. of CSE, ACEIT, Jaipur

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide, **Er. Ram Babu Buri**, for his constant guidance, encouragement and valuable suggestions throughout the development of this project. His insistence on clear requirements, careful design and honest review helped us to understand not only the technologies involved but also the discipline needed to plan a large software system.

We sincerely thank **Dr. Akhil Pandey, Head of the Department of Computer Science and Engineering, Arya College of Engineering & I.T.**, for providing the academic environment and support needed to carry out this work. We are also grateful to all the faculty members of the **Department of Computer Science and Engineering** for their guidance and cooperation.

We extend our thanks to the faculty coordinators and student leads of the fifteen clubs of Arya College, whose experience with organising events, collecting fees, handling venues and issuing certificates shaped the requirements of **CampusSphere**.

We acknowledge the cooperation and team spirit of every member of our group. The exchange of ideas, joint review of specifications and shared problem solving greatly enriched our understanding of team-based software engineering.

Finally, we express our heartfelt gratitude to our families and friends for their constant support, patience and motivation throughout the course of this project.

Govind Jangid (Team Leader)

Garvita Jain

Ayushi Garg

Ankit Yadav

Anshika Pathak

Computer Science & Engineering

ABSTRACT

This report presents the analysis, design and implementation plan of **CampusSphere**, a unified web platform for college events, club finances, venue scheduling, verified attendance, peer feedback and extracurricular credentialing at **Arya College of Engineering & I.T., Jaipur**. The project was motivated by the fragmented way in which fifteen student clubs organise events today: announcements in chat groups, registration through forms, fees sent to personal UPI identifiers, paper attendance sheets, forgeable certificates, unverified reviews and no visibility of the 100 AICTE/RTU activity points that every B.Tech student must earn.

The proposed system offers role-specific dashboards for Students, Club Admins and the Super Admin (Dean/HOD). It provides a seeded directory of fifteen clubs, each with a dedicated treasury ledger; an event proposal and approval workflow; a venue booking engine that enforces a 30-minute setup and 30-minute teardown buffer; atomic seat reservation for solo and team registrations; Razorpay payments verified on the server and credited to the organising club; an anti-screenshot rolling QR pass based on HMAC-SHA256 that changes every 30 seconds; a camera-based gate scanner with a single-scan guarantee; attendance-gated five-factor feedback with a “You Said, We Did” Kanban board; certificates sealed with SHA-256 and publicly verifiable; and an automated activity-points ledger with an official transcript.

The report describes the requirements, a modular-monolith architecture built on React, TypeScript, a stateless JWT and RBAC security layer, and MongoDB with multi-document transactions. It explains the algorithms behind the critical guarantees, including venue collision detection, the atomic capacity lock, idempotent payment settlement and rolling-token validation, and analyses them against a STRIDE threat model. A ten-phase development roadmap with acceptance tests, a risk register and an AI copilot suite and 3D campus digital twin complete the design.

The work is a design and specification study; the performance and adoption figures quoted are targets for a pilot term and not measured results. The project gave the team practical experience of requirement engineering, secure system design, concurrency control and collaborative planning of a large software product.

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION

- 1.1 Background of the Project
- 1.2 About the Institution and Its Clubs
- 1.3 Problem Statement
- 1.4 Objectives of the Project
- 1.5 Scope of the Project
- 1.6 Significance and Expected Benefits
- 1.7 Team Organisation and Working Method
- 1.8 Organisation of the Report

CHAPTER 2 TECHNOLOGIES AND CONCEPTS

- 2.1 React and TypeScript
- 2.2 Vite and Tailwind CSS
- 2.3 Backend: Spring Boot and Node.js
- 2.4 MongoDB and ACID Transactions
- 2.5 JWT Authentication and Refresh Rotation
- 2.6 Password Hashing with BCrypt
- 2.7 HMAC and Time-Based Tokens
- 2.8 SHA-256 Hashing and Digital Seals
- 2.9 Razorpay Payment Gateway
- 2.10 WebSockets and STOMP
- 2.11 PDF Generation
- 2.12 Three.js and React Three Fiber
- 2.13 Artificial Intelligence Components
- 2.14 Progressive Web Applications
- 2.15 Docker and NGINX
- 2.16 Role-Based Access Control

CHAPTER 3 REQUIREMENT ANALYSIS

- 3.1 Existing System and Its Limitations
- 3.2 Proposed System
- 3.3 User Personas
- 3.4 User Classes and Roles
- 3.5 The Fifteen Master Clubs
- 3.6 Functional Requirements
- 3.7 Non-Functional Requirements
- 3.8 Design Constraints
- 3.9 Success Metrics

CHAPTER 4 SYSTEM DESIGN

- 4.1 Architectural Style
- 4.2 Layered Architecture
- 4.3 Request Flow
- 4.4 Database Design
- 4.5 Indexes and Data Integrity
- 4.6 REST API Design
- 4.7 Role-Based Access Control
- 4.8 Event Lifecycle

CHAPTER 5 MODULE DESIGN AND IMPLEMENTATION APPROACH

- 5.1 Identity and Authentication Module
- 5.2 Clubs and Dedicated Treasury Module
- 5.3 Event Proposal and Approval Module
- 5.4 Venue Buffer Conflict Engine
- 5.5 Registration and Atomic Capacity Lock

- 5.6 Team Registration
- 5.7 Dynamic Rolling QR Engine
- 5.8 Live Gate Attendance Scanner
- 5.9 Payments and Split Ledger Module
- 5.10 Verified Feedback and 'You Said, We Did' Module
- 5.11 Verifiable Certificates Module
- 5.12 AICTE Activity Points Engine
- 5.13 AI Copilot Suite
- 5.14 3D Campus Digital Twin

CHAPTER 6 SECURITY ARCHITECTURE

- 6.1 Security Philosophy
- 6.2 Token Lifecycle
- 6.3 Anti-Screenshot Analysis
- 6.4 Payment Security and Split Accounting
- 6.5 Attendance-Gated Feedback Security
- 6.6 Certificate Cryptography
- 6.7 Threat Model (STRIDE)
- 6.8 Privacy and Data Protection
- 6.9 Residual Risks

CHAPTER 7 DEVELOPMENT PLAN, TESTING AND EVALUATION

- 7.1 Phased Development Roadmap
- 7.2 Testing Strategy
- 7.3 Representative Test Cases
- 7.4 Performance Targets and Measurement Plan
- 7.5 Risk Analysis
- 7.6 Observations on the Specification Set
- 7.7 Ethical Considerations

CHAPTER 8 CONCLUSION AND FUTURE SCOPE

- 8.1 Conclusion
- 8.2 Limitations
- 8.3 Future Scope

REFERENCES

CHAPTER 1

INTRODUCTION

1.1 Background of the Project

Extracurricular participation is an essential part of technical education. Hackathons, workshops, coding contests, sports tournaments, cultural festivals and social-service drives develop teamwork, leadership and practical skills that classroom teaching alone cannot provide. Recognising this, the All India Council for Technical Education (AICTE) and the Rajasthan Technical University (RTU) require every undergraduate B.Tech student to earn a minimum of **100 Activity Points** (75 points for lateral-entry students) over the four-year programme in order to qualify for the B.Tech degree with Honours and graduation clearance.

In most colleges, however, the machinery that supports these activities has not kept pace with their scale. Events are announced in chat groups, registrations are gathered through forms, fees are collected on personal payment identifiers, and attendance is marked on paper. Certificates are issued as static images, and students keep them in folders that are often lost long before the final year, when the examination cell asks for proof of points. CampusSphere is the project undertaken by our group to replace this fragmented practice with a single, authenticated and verifiable platform.

1.2 About the Institution and Its Clubs

Arya College of Engineering & I.T. (ACEIT), Jaipur, affiliated to Rajasthan Technical University, Kota, has an active student community organised into fifteen officially recognised clubs. These clubs run events throughout the year and compete for shared campus resources such as the central auditorium, seminar halls and laboratories. The clubs cover technology (Science & Technology, Cipher Coding, ACEIT Hackathon, Drones, Robotics, Automation, IoT and LINCOM), culture (Dance, Music, Literature), sports and mind games (E-Sports, Chess) and social responsibility (Social Activities, Green Energy). Because the clubs differ so widely in size, event type and funding, a platform for the college has to be flexible enough for a thirty-six-hour hackathon with paid tickets as well as a free blood-donation drive.

1.3 Problem Statement

A study of current practice at the college identified the following recurring problems:

- **Fragmented communication:** announcements and updates are lost in chaotic messaging groups.

- **Unvalidated registration:** online forms accept duplicate and fake entries and create manual spreadsheet work for club leads.
- **No financial auditability:** fees are sent to personal UPI identifiers of coordinators, so club money is commingled with personal funds and cannot be audited by the Dean.
- **Weak attendance control:** paper lists and static QR codes can be bypassed by forwarded screenshots and proxy signatures.
- **Unverified feedback:** anyone can post a review, so organisers receive spam while genuine suggestions go unanswered.
- **Forgeable certificates:** a static image certificate can be edited using common graphics software.
- **Invisible activity points:** students cannot see how many AICTE/RTU points they have earned, and HODs face a large manual verification burden at the end of the course.
- **Venue clashes:** two clubs can unknowingly book the same hall, or book back-to-back without time for setup and cleanup.

1.4 Objectives of the Project

The project was undertaken with the following objectives:

- To design a unified digital platform that covers the full lifecycle of a college event: proposal, approval, venue booking, registration, payment, attendance, feedback, certification and credit of activity points.
- To provide role-specific dashboards for the three user groups: Student, Club Admin and Super Admin (Dean/HOD).
- To give every club a dedicated treasury (sub-ledger) so that event fees are credited to the club and remain visible to the college for audit.
- To eliminate screenshot-based proxy entry using a rolling cryptographic QR pass that changes every 30 seconds.
- To guarantee that reviews, certificates and activity points can only be obtained by students whose attendance was verified at the venue gate.
- To issue certificates whose authenticity can be verified publicly using a SHA-256 seal.

- To automate the AICTE/RTU activity-point ledger and produce an official transcript.
- To add intelligence through a recommender, an event copilot, sentiment analysis and a campus chatbot, and to present the campus as an interactive 3D digital twin.

1.5 Scope of the Project

The scope covers thirteen functional subsystems: identity and role management, the fifteen-club directory with treasuries, the event engine, venue booking with buffers, solo and team registration, payments and ledgers, the dynamic QR engine, the live gate scanner, verified feedback with a Kanban board, certificates, activity points, the AI copilot suite and the 3D campus twin. The platform is designed as a responsive progressive web application so that it works on Android, iOS and desktop browsers without a separate app installation. Features outside the scope include direct bank payouts to club accounts (settlement is tracked but executed by the college accounts office), integration with the university examination system, and native mobile applications.

1.6 Significance and Expected Benefits

Table 1.1: Expected benefits by stakeholder

Stakeholder	Expected benefit
Students	Single place to discover events, pay securely, hold a tamper-resistant pass, see activity points in real time, and share verifiable certificates on professional networks.
Club leads	Clash-free venue booking, automatic fee collection into the club ledger, fast QR scanning at the gate, and structured, verified feedback.
Dean / HOD	Central approval queue, campus-wide financial audit, verified attendance records and an official transcript, reducing manual verification effort.
College	A durable, auditable record of extracurricular activity that supports accreditation and student placement.

1.7 Team Organisation and Working Method

The project was carried out by a group of five students: **Govind Jangid (Team Leader)**, **Garvita Jain**, **Ayushi Garg**, **Ankit Yadav** and **Anshika Pathak**, under the guidance of **Er. Ram Babu Buri**. The group followed a phased, module-by-module approach. For each module the team first fixed the domain model and the rules, then designed the service and API contracts, then planned unit and integration tests, and finally designed the user interface. A specification suite was prepared first and reviewed jointly so that every member worked from the same agreed contracts. The suite comprises the Product Requirements Document (PRD), the Software Requirements Specification (SRS), the Entity Relationship Diagram

(ERD), the Technical Requirements Document (TRD), the REST API Specification, the Security Architecture, the AICTE Activity Points Specification and the Development Roadmap.

1.8 Organisation of the Report

Chapter 2 introduces the technologies and concepts used. Chapter 3 presents requirement analysis. Chapter 4 describes the system design: architecture, data model, API and access control. Chapter 5 explains each module and the algorithms behind it. Chapter 6 discusses the security architecture. Chapter 7 gives the development plan, testing strategy and risk analysis, and Chapter 8 concludes with future scope.

CHAPTER 2

TECHNOLOGIES AND CONCEPTS

This chapter summarises the technologies and ideas on which CampusSphere is built. The selection was made on the criteria of type safety, developer productivity, security, scalability and suitability for a student-run, institution-hosted service.

2.1 React and TypeScript

React is a component-based JavaScript library for building user interfaces. A screen is composed of small reusable components, and the interface updates efficiently when data changes. CampusSphere uses React 19 with TypeScript, a typed superset of JavaScript. Typed interfaces for entities such as User, Club, Event, Venue, Registration, Payment, Attendance, Feedback and Certificate are shared between modules, so that a wrong field name or a missing property is caught while writing code rather than at run time.

2.2 Vite and Tailwind CSS

Vite is a modern build tool that provides very fast start-up and hot module replacement during development and optimised bundles for production. Tailwind CSS is a utility-first styling framework that allows a consistent, responsive design with dark and light themes, which is important because the application must work on small phones at the venue gate as well as on desktop dashboards.

2.3 Backend: Spring Boot and Node.js

The backend exposes a stateless REST API. The architecture documents allow two equivalent implementations: Spring Boot 3.3 on Java 21 with Spring Security, or Node.js with Express or NestJS. Both provide dependency injection, validation, middleware for authentication and well-developed libraries for MongoDB and payment gateways. The data model and the API contract are identical in both, so the team can choose according to available skills without changing the design.

2.4 MongoDB and ACID Transactions

MongoDB is a document database that stores records as JSON-like documents. It suits event feeds, which are read-heavy and have flexible fields. Since version 4.0 MongoDB has supported multi-document ACID transactions on replica sets, which CampusSphere uses at the three places where several documents must change together: confirming a payment,

recording a gate scan and crediting activity points. For everything else the single-document atomic operators such as `$inc` and `$set` are sufficient and fast.

2.5 JWT Authentication and Refresh Rotation

A JSON Web Token (JWT) is a compact, signed token that carries claims such as the user identifier and role. CampusSphere issues a short-lived access token (15 minutes) and a long-lived refresh token (7 days). The refresh token is stored hashed in the database and delivered only through an `HttpOnly`, `Secure`, `SameSite=Strict` cookie so that scripts cannot read it. On each refresh the token is rotated, and it is revoked on logout or password change.

2.6 Password Hashing with BCrypt

Passwords are never stored in readable form. BCrypt is an adaptive hashing function that is deliberately slow and includes a random salt, which makes brute-force and rainbow-table attacks costly. The platform uses a work factor (cost) of 12.

2.7 HMAC and Time-Based Tokens

A Hash-based Message Authentication Code (HMAC) combines a secret key with a hash function (here SHA-256) to produce a signature that only the holder of the key can compute. The Time-based One-Time Password (TOTP) algorithm applies this idea to a time counter so that codes expire automatically. CampusSphere adapts it to build a rolling QR pass: the code changes every 30 seconds, and a screenshot becomes useless once its window has passed.

2.8 SHA-256 Hashing and Digital Seals

SHA-256 is a cryptographic hash function that maps any input to a 256-bit digest. Even a one-character change in the input produces a completely different digest. Each certificate carries a seal computed from its identifier, the student roll number, the event identifier, the issue date and a private college salt, so any edit to the printed certificate can be detected by recomputation.

2.9 Razorpay Payment Gateway

Razorpay is an Indian payment gateway that supports UPI, cards and net banking. The merchant server creates an order for a fixed amount, the student pays in the gateway checkout, and the gateway returns identifiers and a signature that the server must verify. Razorpay also sends asynchronous webhook events such as `payment.captured`, signed with a webhook secret, which makes confirmation reliable even if the student closes the browser early.

2.10 WebSockets and STOMP

WebSockets provide a persistent two-way connection between browser and server. CampusSphere uses it (with the STOMP messaging protocol in the Spring variant) to push live gate-entry counts, ticket confirmations and emergency broadcast alerts without polling.

2.11 PDF Generation

Certificates and the activity-points transcript are generated as vector PDFs using PDFKit (Node.js) or iText 8 (Java). Vector output keeps crests, signatures and text sharp at any print size, and allows a QR code to be embedded for verification.

2.12 Three.js and React Three Fiber

Three.js is a WebGL library for 3D graphics in the browser, and React Three Fiber lets 3D scenes be described as React components. The campus digital twin uses a low-poly model, instancing and simple materials so that it stays above 50 frames per second on phones with an asset bundle below 2 MB.

2.13 Artificial Intelligence Components

The AI suite relies on large language models (LLMs) for text generation and conversation, embeddings for turning events and interests into numeric vectors that can be compared by similarity, and natural-language-processing techniques for sentiment scoring and clustering similar suggestions. A retrieval-augmented generation (RAG) approach grounds the chatbot's answers in platform data about events, venues and club rules, which reduces made-up answers.

2.14 Progressive Web Applications

A Progressive Web App (PWA) is a web application that can be installed to the home screen, works on unreliable networks through caching and can access device features such as the camera. This is the basis for the in-browser QR scanner used by Club Admins at the gate.

2.15 Docker and NGINX

Docker packages the application, database and proxy into containers so that the same stack runs identically on a laptop and on the college server. NGINX acts as the reverse proxy: it terminates TLS, compresses static assets and applies token-bucket rate limiting of 100 requests per minute in general and 5 per minute on authentication endpoints.

2.16 Role-Based Access Control

Role-Based Access Control (RBAC) assigns permissions to roles instead of to individual users. CampusSphere defines three roles, STUDENT, CLUB_ADMIN and SUPER_ADMIN, and enforces them in the service layer on every operation, never relying only on the user interface to hide buttons.

Table 2.1: Technology stack summary

Layer	Technology	Reason for selection
Frontend	React 19, TypeScript, Vite, Tailwind CSS	Type safety, fast builds, responsive utility styling
3D	Three.js, React Three Fiber	WebGL rendering of the campus twin
State and cache	TanStack Query v5, Context	Client-side caching and revalidation
Backend	Spring Boot 3.3 (Java 21) or Node.js with Express/NestJS	Robust security ecosystem and validation
Database	MongoDB 7 replica set	Document flexibility with ACID transactions
Payments	Razorpay SDK	UPI/card gateway with signed webhooks
Real time	WebSocket / STOMP	Live venue alerts and counters
AI	LLM API, vector embeddings	Recommendation, copywriting, NLP
PDF	PDFKit / iText 8	Vector certificates and transcripts
Deployment	Docker, NGINX	Reproducible deployment, TLS, rate limiting

CHAPTER 3

REQUIREMENT ANALYSIS

3.1 Existing System and Its Limitations

At present, every club at the college manages its own events with its own tools. A typical event begins with a poster shared in messaging groups. Registration happens through an online form that exports to a spreadsheet; fees, if any, are sent to the personal UPI address of a coordinator, and the coordinator later reconciles payments manually against the form responses. At the venue, volunteers tick names on printed lists. After the event the club may circulate a feedback form and prepares certificates in a graphics tool. The Dean's office has no consolidated view of activity, finance or points.

Table 3.1: Limitations of the existing system

Area	Limitation
Communication	No single authoritative channel; updates are lost
Registration	Duplicate entries, no capacity control, manual reconciliation
Finance	Funds routed to personal accounts; no audit trail
Attendance	Paper sheets; screenshots and proxies bypass checks
Feedback	Anyone can submit; no link to actual attendance
Certificates	Static images; easily forged; often lost
Activity points	Not tracked; disputes at graduation time
Venue	No clash detection; no setup/teardown buffer

3.2 Proposed System

CampusSphere consolidates these steps into one authenticated platform with role-specific dashboards. Its core idea is that each downstream benefit is gated by an upstream verified fact. A paid registration yields a ticket only after server-side payment verification. A scanned, valid, single-use QR pass at the gate creates an attendance record. Only a verified attendance record unlocks feedback, a certificate and activity points, and only after the event has been marked COMPLETED.

3.3 User Personas

Table 3.2: Target personas

Persona	Goals	Pain points
Rohan, 3rd-year CSE student	Discover hackathons, register with a team, pay securely, show an instant pass, claim activity points and LinkedIn-ready certificates	Misses deadlines in chat groups; forwarded fake tickets; no view of total points

Persona	Goals	Pain points
Priya, President of the Cipher Coding Club	Create event pages, check venue availability, collect fees, scan passes quickly, gather feedback	Manual UPI reconciliation; venue clashes; fake attendance claims; reviews from non-attendees
Dr. Sharma, Dean / HOD	Oversee clubs, approve or reject proposals, audit finances, resolve clashes, verify transcripts	No central record of club funds; surprise clashes in the auditorium; certificate disputes

3.4 User Classes and Roles

Three user classes are supported. The **STUDENT** discovers events, joins teams, pays, shows the rolling QR pass, earns points, downloads certificates and submits verified feedback. The **CLUB_ADMIN**, a club executive or faculty coordinator, drafts events, books venues, views registrations, operates the gate scanner, tracks the club treasury in read-only mode and manages the 'You Said, We Did' board. The **SUPER_ADMIN**, the Dean, HOD or Principal, approves or rejects events, resolves venue clashes, audits all club ledgers and signs off activity-point transcripts.

3.5 The Fifteen Master Clubs

Table 3.3: Master club directory

No.	Club	Identifier	Focus
1	Arya Science & Technology Club	arya_scitech	Exhibits, tech fairs, patents
2	Arya Cipher Coding Club	arya_cipher	Competitive coding, web/app
3	Arya ACEIT Hackathon Club	arya_aceit_hack	24-36 h hackathons
4	Arya E-Sports Club	arya_esports	Valorant, BGMI, FIFA
5	Arya Dance Club	arya_dance	Classical to hip-hop
6	Arya Social Activities Club	arya_social	Blood drives, CSR
7	Arya Literature Club	arya_lit	Debates, MUN, writing
8	Arya Drones Club	arya_drones	Drone building, FPV racing
9	Robotics Club	arya_robotics	Robo-wars, rovers
10	Automation Club	arya_automation	Industrial IoT, PLC
11	Green Energy Club	arya_green	Solar, EV, sustainability
12	Music Club	arya_music	Bands, acoustic sessions
13	Chess Club	arya_chess	FIDE-rated tournaments
14	IoT Club / Arya Intelverse	arya_iot	AIoT, microcontrollers
15	LINCOM, Arya Linux Community	arya_lincom	Linux, FOSS, Git

3.6 Functional Requirements

The Software Requirements Specification, written in IEEE Std 830 format, defines thirteen groups of functional requirements. They are summarised in Table 3.4.

Table 3.4: Summary of functional requirement groups

ID	Group	Key requirements
FR-1	Authentication and roles	Registration with roll number and institutional email; BCrypt cost 12; 15-minute access and 7-day refresh tokens; role-based authorisation
FR-2	Clubs and treasury	15 seeded clubs; isolated sub-ledger with total revenue, available balance, pending settlement and payout destination
FR-3	Event lifecycle	Draft, PENDING_APPROVAL, approve/reject with notes, then LIVE and COMPLETED
FR-4	Venue booking	Capacity limits; 30-minute setup and 30-minute teardown buffers; automatic conflict alerts
FR-5	Registration	Atomic seat reservation; team passcodes; unique (event, user) index
FR-6	Payments	Razorpay order per club; server HMAC verification; immutable ledger credit
FR-7	Rolling QR	30-second HMAC token, countdown ring, holographic watermark
FR-8	Gate scanner	Camera scan; signature, event and ticket checks; duplicate red alert
FR-9	Feedback	Five-factor ratings only after verified attendance and COMPLETED status; suggestion board and Kanban
FR-10	Certificates	PDF with serial and SHA-256 seal; public verification; add-to-LinkedIn
FR-11	Activity points	Automatic credit; four domains; official transcript PDF with QR
FR-12	AI suite	Recommender, event copilot, sentiment clustering, concierge chatbot
FR-13	3D twin	Low-poly campus, pulsing beacons for live events, event drawer

3.7 Non-Functional Requirements

Table 3.5: Non-functional requirements

Category	Requirement
Performance	Event discovery under 200 ms for up to 10,000 events; gate validation under 400 ms on 4G; 3D scene above 50 FPS and under 2 MB
Security	TLS 1.3; authorisation on every service method; webhook signature validation; rate limits 100 requests/min (5 for authentication)
Reliability	99.9% uptime during peak fest hours; concurrency-safe booking with no oversubscription

Category	Requirement
Compatibility	Responsive on iOS and Android browsers, tablets and desktop; modern browsers with WebGL
Integrity	Atomic \$inc seat reservation; indexes on email, roll number, slug, ticket number and compound keys
Auditability	Immutable audit log for approvals, payments, cancellations and manual overrides

3.8 Design Constraints

- **Immutable accounting:** Club Admins must never be able to alter ledger balances directly; balances change only through backend transaction events.
- **Attendance gating:** feedback must be impossible without a verified attendance record created at the gate.
- **No screenshot forwarding:** the QR token must change every 30 seconds.
- **Zero trust in the client:** price, status and role are always decided on the server.

3.9 Success Metrics

Table 3.6: Key performance indicators

Dimension	Target
Engagement	More than 80% of the student population registered in the first semester after launch
Attendance accuracy	0% duplicate gate check-ins or screenshot replay successes
Feedback rate	More than 60% attendance-verified review completion
Financial transparency	100% of registration fees tracked in dedicated club balances
Administrative efficiency	Venue approval time reduced from 3 business days to under 2 hours

These indicators are targets for the pilot term after launch; they are not results measured in this report.

CHAPTER 4

SYSTEM DESIGN

4.1 Architectural Style

CampusSphere follows a domain-driven modular monolith architecture. The application is a single deployable unit, but is organised internally into bounded modules with explicit service contracts, so that a module such as payments or the AI suite can later be extracted into an independent service without redesign. This choice was made for four reasons: it gives rapid development and single-command deployment; it avoids the overhead of distributed transactions, which matter because payment confirmation, seat reservation and attendance each need multi-document atomicity; it keeps module boundaries strict; and it is manageable for a student team.

The core tenets are: (1) a **stateless security perimeter**, in which no session state is kept on a server instance and authorisation is driven by signed tokens; (2) **ACID transactions** on critical boundaries; (3) **idempotency by design** for financial webhooks and ticket issuance; (4) **offline resilience**, in which gate scanners can validate dynamic passes locally even when venue connectivity is unstable; and (5) **separation of concerns**, with client business logic isolated in hooks and services and backend domains communicating through service contracts.

4.2 Layered Architecture

Figure 4.1 shows the layers. A request travels from the browser or mobile PWA through the NGINX reverse proxy, which terminates TLS and applies rate limiting. The API gateway and security layer then authenticates the bearer token, applies CORS rules for the permitted Arya College domains and checks method-level authority. REST controllers receive the request and delegate to service classes, and a WebSocket dispatcher pushes live events such as check-ins. The service layer contains the domain services: Event Engine, Venue Buffer, Dynamic QR, Ledger, Gate Scanner, Feedback Gate, AICTE Points and AI Copilot. These talk to MongoDB, to external gateways (Razorpay and S3/Cloudinary storage) and to the AI inference engine.

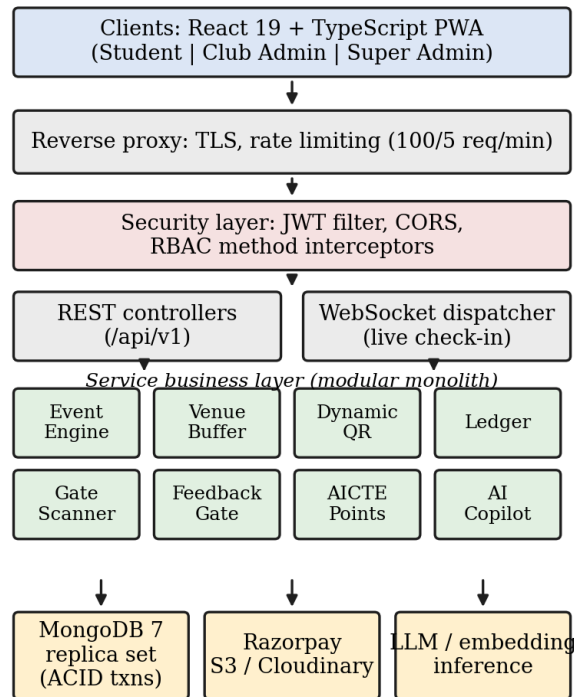


Figure 4.1: Layered architecture of CampusSphere

4.3 Request Flow

Controllers contain no business rules. On each request the JWT filter extracts the user identifier, role and administered clubs from the token, a method-level interceptor checks the required authority (for example `hasAuthority('SUPER_ADMIN')` for event approval), and the controller calls a service. The service performs validation and data access, writes an audit record for sensitive operations, and returns a result that the controller wraps in the standard response envelope.

4.4 Database Design

The database uses a hybrid modelling strategy tuned for high-read event feeds and strict correctness for money and attendance. Data that is tightly coupled and rarely read alone is **embedded**, for example a student's academic profile inside the user document, venue facilities, social links and the ticket's cryptographic seed. High-cardinality or independently queried entities are **referenced**: events, registrations, payments, attendance records, club ledgers, feedback and certificates. Atomic document-level operators (`$inc`, `$set`, `$addToSet`) prevent race conditions during rapid ticket booking without heavy locking.

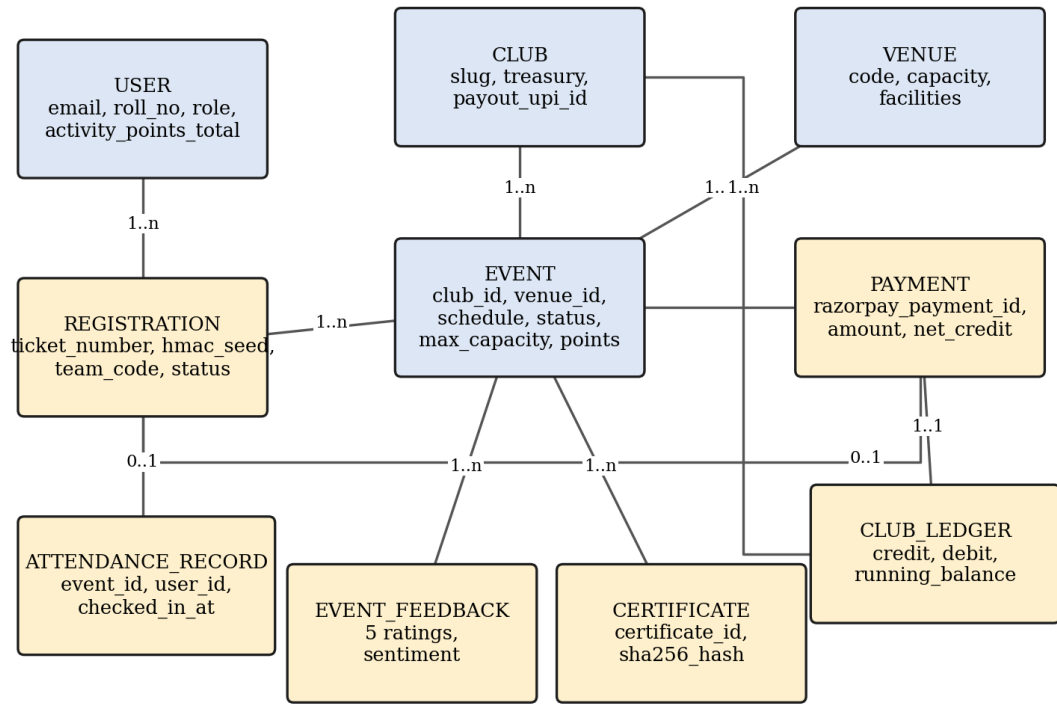


Figure 4.2: Entity relationship overview of CampusSphere

Table 4.1: Principal collections

Collection	Important fields and relationships
users	email (unique), password_hash, role, name, roll_no (unique), department, semester, activity_points_total, interests
clubs	slug (unique), name, category, description, logo_url, faculty_coordinator, total_revenue, available_balance, payout_upi_id
venues	code (unique), name, capacity, location, facilities, is_active
events	club_id, venue_id, slug (unique), title, category, tags, start_time, end_time, registration_type, is_paid, ticket_price, max_capacity, registered_count, activity_points, status
registrations	event_id, user_id, payment_id, team_name, team_code, status, ticket_number (unique), hmac_seed, attendance_verified
payments	registration_id, event_id, club_id, payer_user_id, amount_inr, net_credited_to_club, razorpay_order_id, razorpay_payment_id (unique), status
club_ledgers	club_id, payment_id, event_id, transaction_type, credit_amount, debit_amount, running_balance, remarks, timestamp
attendance_records	event_id, user_id, ticket_number, checked_in_at, scanner identity
event_feedback	event_id, user_id, five ratings, review text, sentiment label and score
suggestions	event_id, club_id, author, text, upvotes, Kanban status
certificates	certificate_id, sha256_hash, user_id, event_id, issued_at
audit_logs	user_id, ip_address, action, diff, timestamp (365-day TTL)

4.5 Indexes and Data Integrity

Several invariants are enforced by unique indexes rather than by application code, because an index cannot be bypassed by a race between two requests. Table 4.2 lists them.

Table 4.2: Indexes and the invariants they protect

Index	Invariant protected
users(email), users(roll_no) unique	One account per identity
events(slug) unique	Stable public event URLs
events(venue_id, start_time, end_time)	Fast venue-buffer collision scan
registrations(event_id, user_id) unique	No duplicate registration
registrations(ticket_number) unique	Unique ticket identity
attendance_records(event_id, user_id) unique	Single check-in guarantee
payments(razorpay_payment_id) unique	No double credit of a payment
event_feedback(event_id, user_id) unique	One review per attendee per event
certificates(certificate_id) unique	Verifiable serial number

4.6 REST API Design

The API is served under the versioned base path `/api/v1` and uses JSON. Authentication is through an HTTP bearer header carrying the access token. Every response uses a common envelope with the fields `success`, `status_code`, `message`, `data` and `timestamp`. Errors replace `data` with a machine-readable `error_code` and an `errors` array.

```
{ "success": true,
  "status_code": 200,
  "message": "Operation successful",
  "data": { ... },
  "timestamp": "2026-03-15T10:00:00.000Z" }
```

Table 4.3: Representative REST endpoints

Endpoint	Access	Purpose
POST /auth/register	Public	Register a student with roll number, department, semester and interests
POST /auth/login	Public	Return access token; set HttpOnly refresh cookie
POST /auth/refresh	Public (cookie)	Rotate the refresh token
GET /clubs	Public	List all 15 clubs, filter by category

Endpoint	Access	Purpose
GET /clubs/:id/treasury	Club Admin (own) / Super Admin	Read-only treasury and recent ledger entries
POST /venues/check-conflict	Authenticated	Check availability with 30-minute buffers; return alternatives
POST /events	Club Admin	Submit an event proposal (PENDING_APPROVAL)
PATCH /events/:id/status	Super Admin	Approve or reject with comments
POST /registrations/solo	Student	Reserve a seat atomically
GET /registrations/:id/dynamic-qr	Ticket owner / Club Admin	Return the current 30-second signed token
POST /payments/create-order	Student	Create a Razorpay order (server-calculated amount)
POST /payments/verify-signature	Student	Verify signature and issue ticket
POST /attendance/verify-scan	Club Admin / Super Admin	Validate a scanned pass at the gate
POST /feedback/submit	Student (verified)	Submit five-factor feedback
POST /suggestions/:id/upvote	Student	Upvote a suggestion
GET /certificates/verify/:certId	Public	Verify a certificate without logging in

Useful error codes are part of the contract: `VENUE_BUFFER_COLLISION` (409) returns the conflicting event and free alternatives, `DUPLICATE_CHECK_IN` (409) returns the first check-in time, and `VERIFIED_ATTENDANCE_REQUIRED` (403) blocks feedback from students who were not scanned at the gate.

4.7 Role-Based Access Control

The roles form a hierarchy in which the `SUPER_ADMIN` holds full campus oversight and the `CLUB_ADMIN` and `STUDENT` roles hold narrower authorities. Every service method checks the caller's authority, and club-scoped operations additionally check that the club identifier is in the `clubs` claim of the token.

Table 4.4: Authority matrix

Role	Authority
<code>SUPER_ADMIN</code>	Full campus oversight; event approval and rejection; global financial audit; venue conflict overrides; AICTE transcript sign-off
<code>CLUB_ADMIN</code>	Create and edit own-club events; book venues; launch the gate QR scanner; view own club treasury (read-only); manage suggestions on the Kanban board
<code>STUDENT</code>	Register solo or in teams; pay; view personal rolling QR tickets; check attendance; claim certificates; submit verified feedback

4.8 Event Lifecycle

An event is created as a draft by a Club Admin, submitted for approval, and then either approved or rejected by the Super Admin with mandatory review notes. Approved events become LIVE at the scheduled time and COMPLETED when they end. The transition to COMPLETED triggers the background jobs for certificates and activity points (Figure 4.3).

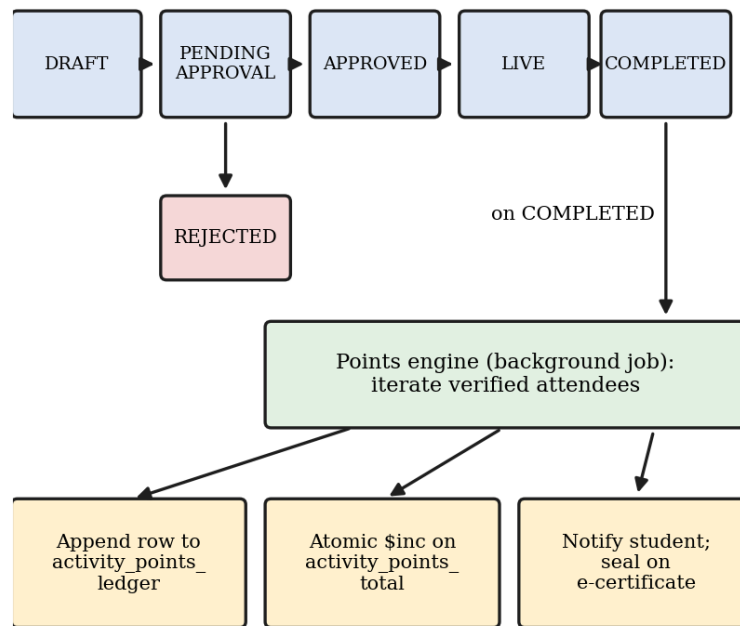


Figure 4.3: Event lifecycle and the points-crediting job

CHAPTER 5

MODULE DESIGN AND IMPLEMENTATION APPROACH

This chapter explains how each functional module is designed, including the algorithms that implement the critical guarantees. The descriptions follow the specification suite and the technical requirements document.

5.1 Identity and Authentication Module

Students register with their Arya College roll number (for example 22EACIT089), institutional email, department, semester and interests. Passwords are hashed with BCrypt (cost 12). On login the server returns the access token in the response body and sets the refresh token in an HttpOnly cookie. The role field is stripped from public registration and profile-update objects, which removes the simplest privilege-escalation path. Access tokens carry the claims `sub`, `role`, `roll`, `clubs`, `exp` and `iat` and are signed with HS512 or RS256.

5.2 Clubs and Dedicated Treasury Module

A database seeder creates all fifteen clubs with logos, faculty leads and student coordinators. Each club owns an isolated sub-ledger holding total revenue, available balance, pending settlement and the payout UPI identifier. A Club Admin can see only the assigned club, and the treasury is read-only: there is no endpoint that updates `available_balance` directly. Balances change only as a result of backend transaction events, and each change appends a ledger row with the type, credit or debit, running balance and remarks, for example a ticket sale linked to a ticket number.

5.3 Event Proposal and Approval Module

The event wizard collects the title, category, tags, venue, schedule, Solo or Team mode, free or paid pricing, maximum capacity and the proposed AICTE points. On submission the event enters `PENDING_APPROVAL` and appears in the Super Admin's queue. The approver can approve or reject with comments, and every decision is written to the audit log. Because the points are part of the proposal, the weightage is reviewed by the Dean before students can register.

5.4 Venue Buffer Conflict Engine

Campus venues such as the main auditorium, Seminar Halls A and B, the central lawn and laboratories need time for sound setup, lighting and cleanup. For a proposed booking with start time s and end time e , the engine extends the window by the buffer:

$$W = [s - 30 \text{ min}, e + 30 \text{ min}] \quad (1)$$

It then searches approved or live events at the same venue whose own start time is before $W.end$ and whose end time is after $W.start$. If any exist, the booking is rejected and the response lists the conflicting event and available alternative venues.

```
{ "venue_id": proposed_venue_id,
  "status": { "$in": ["APPROVED", "LIVE"] },
  "event_schedule.start_time": { "$lt": W.end },
  "event_schedule.end_time": { "$gt": W.start } }
```

Since the extended window of the new event is compared with the raw times of existing events, any two events at a venue are separated by at least 30 minutes, which is shared between the teardown of the first and the setup of the second. For example, an event ending at 08:45 conflicts with a new event starting at 09:00, because the new window begins at 08:30. To prevent two clubs from booking the same slot at the same instant, the check and the approval should run inside one transaction or use a conditional write on a per-venue document.

5.5 Registration and Atomic Capacity Lock

Flash registrations for popular events create a classic race: two students may both read that one seat remains and both book it. CampusSphere avoids this by making the capacity check and the increment one atomic database operation:

```
db.events.updateOne(
  { _id: eventId, status: 'APPROVED',
    'ticketing.registered_count': { $lt: max_capacity } },
  { $inc: { 'ticketing.registered_count': 1 } })
// matchedCount === 0 -> sold out (waitlist)
```

Because a single-document update in MongoDB is atomic, at most C concurrent requests can succeed for an event of capacity C ; oversubscription is impossible by construction. The counter update and the creation of the registration document must be tied together, either in a transaction or with a compensating decrement if the second write fails, otherwise a failed write would permanently consume a seat. For paid events the seat is held while payment is in progress and released by a scheduled job if payment is not completed within a timeout such as ten minutes.

5.6 Team Registration

Hackathon and esports events are team-based. A captain names the squad and receives a passcode such as ACE-789X. Teammates join by entering the passcode, and the registration

finalises once the minimum squad size, for example two members, is reached. Four alphanumeric characters give $36^4 = 1,679,616$ suffixes per prefix. For n active teams the chance of an accidental duplicate when codes are drawn at random is approximately $n^2 / (2 \times 36^4)$, which is about 0.3% for 100 teams. The generator therefore checks uniqueness inside the event and retries. A passcode is an invitation token rather than a secret, so joining still requires an authenticated student who has not already registered for the event.

5.7 Dynamic Rolling QR Engine

Every confirmed registration stores a random 256-bit seed K . The phone derives a time window and a signature:

$$T = \text{floor}(t / 30) \quad (2)$$

$$\text{sig} = \text{HMAC-SHA256}(K, T \parallel \text{ticket_number} \parallel \text{user_id}) \quad (3)$$

The QR code encodes the Base64URL form of the payload $\{ \text{tk}, \text{uid}, \text{eid}, \text{t}, \text{sig} \}$. The screen shows the code with an animated circular ring counting down from 30 to 0 seconds and a shimmering holographic watermark that is hard to reproduce in a still image. A new code is generated each 30 seconds. A screenshot is useful only until the scanner's time window moves beyond the accepted range, and a forwarded image cannot win entry if the real holder has already been scanned.

5.8 Live Gate Attendance Scanner

Club Admins open the scanner in the browser, which uses the phone or laptop camera. The validation pipeline (Figure 5.1) rejects a scan for the wrong event, checks the time window against $T-1$, T and $T+1$ to tolerate clock differences between phones, recomputes the HMAC with the stored seed, and finally checks the attendance collection for an existing record of the same event and user. If a record exists, the scanner shows a loud, high-visibility red banner and sounds an alarm with the original check-in time, for example 'Already Checked-In at 09:14 AM - Duplicate Rejected'. Otherwise an attendance record is written atomically, the live counter is incremented and the new count is pushed through the WebSocket channel.

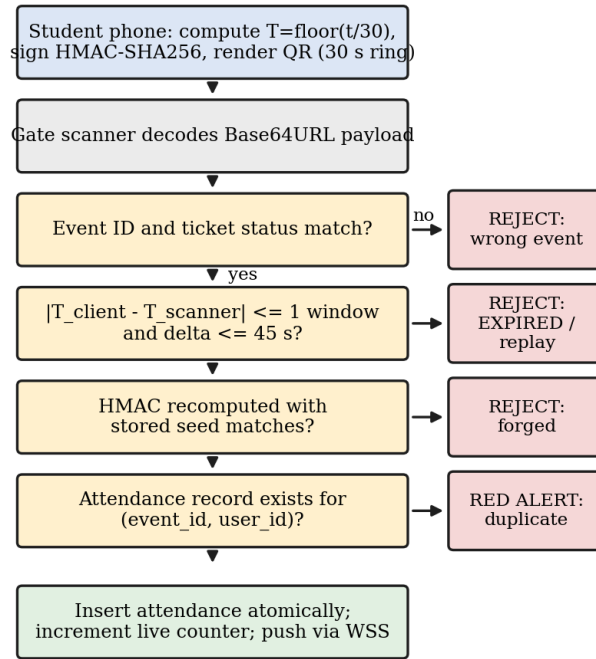


Figure 5.1: Gate-scan validation pipeline

Replay window analysis. Because a scanner in window S accepts token windows $S-I$ to $S+I$, one token can be acceptable over a span of up to 90 seconds, although its intended life is 30 seconds. The technical document also rejects any timestamp difference above 45 seconds. Adopting the stricter 45-second limit as the single rule keeps the exposure below the 90-second span. With a 256-bit signature the probability of guessing a valid token is 2 to the power of minus 256 per attempt, which is negligible.

Offline validation. For venues with poor connectivity the scanner needs the seed, or a key derived from it, on the device. Distributing per-ticket seeds to many devices enlarges the attack surface, so a per-event derived key, with per-ticket seeds derived from it, would restrict exposure to a single event.

5.9 Payments and Split Ledger Module

Paid events are handled as shown in Figure 5.2. The server, not the client, determines the price and creates a Razorpay order (for example 25,000 paise for INR 250). After the student pays in the gateway checkout, the client returns the order identifier, payment identifier and signature, and the server recomputes:

$$expected = HMAC-SHA256(SECRET, order_id \mid payment_id) \quad (4)$$

If the signature does not match, the request is rejected and logged as a fraud attempt. The asynchronous webhook `payment.captured` is handled in the same way, with its `X-Razorpay-Signature` header validated against the webhook secret. Before processing, the system checks whether the payment identifier already exists and, if so, returns HTTP 200 without any change. Otherwise, in one ACID transaction the system sets the registration to `CONFIRMED`, generates the ticket number and seed, creates the payment record with the club identifier, appends a credit to the club ledger and updates the club's available balance. Since both the client verification and the webhook may arrive for the same payment, the unique index on the payment identifier is the final arbiter and double credit cannot occur.

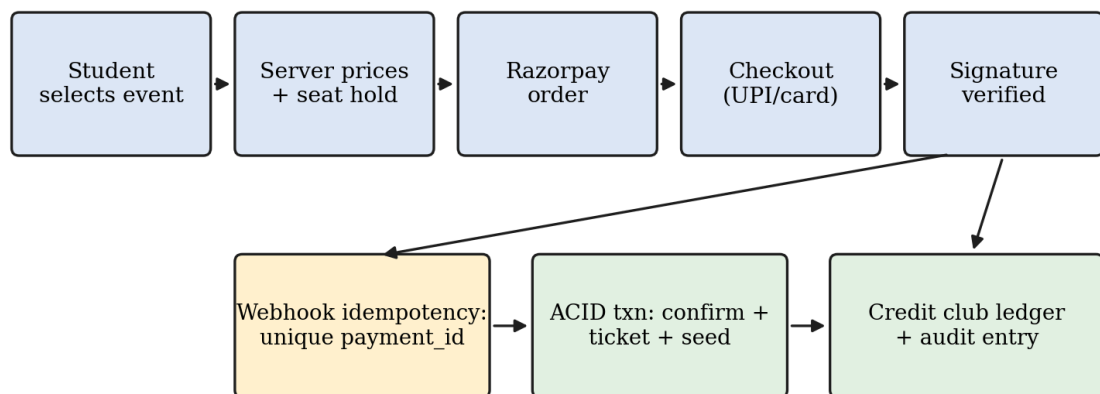


Figure 5.2: Payment verification and ledger settlement flow

5.10 Verified Feedback and 'You Said, We Did' Module

Feedback is gated by four checks, executed in this order: the student must hold a registration (otherwise 403), the event must be `COMPLETED` (otherwise 400), a gate-created attendance record must exist (otherwise 403), and no earlier review may exist (otherwise 409). Only then are five ratings (content depth, organisation, speaker quality, venue facilities and value for time) stored along with an overall rating, the free text is analysed for sentiment and the review is displayed with a Verified Attendee badge. The unique index on (event, user) makes the last check free of races.

Verified attendees can also post public suggestions and upvote the ideas of others. Club Admins move suggestions across a public Kanban board through the stages `SUBMITTED`, `UNDER_REVIEW`, `PLANNED` and `IMPLEMENTED`, attaching notes or photographs as proof. This closes the feedback loop visibly and encourages students to participate.

5.11 Verifiable Certificates Module

After an event is completed, a certificate is generated for each verified attendee. Each certificate has a serial number such as CS-ARYA-2026-HACK-0142 and a seal:

$$Seal = SHA-256(CertificateID || RollNo || EventID || IssueDate || COLLEGE_PRIVATE_SALT) \quad (5)$$

The seal is stored in the database and printed on the PDF as a QR code and an alphanumeric code. When a recruiter scans the QR or opens `/verify/:certId`, the backend recomputes the hash and displays the student name and roll number, the event title and organising club, the points awarded, the issue date and the Dean's verification seal. If the PDF has been altered in a graphics tool, the recomputed values do not match the stored record. The student interface also offers a one-click 'Add to LinkedIn Licenses & Certifications' button pre-filled with the certificate metadata.

5.12 AICTE Activity Points Engine

Each event carries a category code and a point value approved by the Super Admin. When an event is marked COMPLETED, a background job loops through the verified attendees, appends a row to the activity points ledger, atomically increments `activity_points_total` on each student, sends a push notification such as 'Congratulations! You earned 25 AICTE Activity Points.' and embeds the point seal in the certificate. Table 5.1 presents the approved weightage matrix.

Table 5.1: Activity point matrix by event category

Category code	Event types	Eligible clubs	Points
ACT_TECH_HACK	24-36 h national hackathons	ACEIT Hackathon, Cipher	25
ACT_TECH_WKS	Bootcamps and workshops over 8 h	Cipher, Drones, Robotics, Automation, IoT	15
ACT_TECH_COMP	Coding contests, robo-wars, FPV racing	SciTech, LINCOM, Robotics, Drones	10
ACT_SOC_CSR	Blood donation, rural tech camps	Social Activities	20
ACT_SOC_ENV	Tree plantation, solar energy expo	Green Energy, Social	15
ACT_LIT_DEB	MUN, parliamentary debate, elocution	Literature	15
ACT_CULT_FEST	Inter-college fest, band/dance competitions	Dance, Music	15
ACT_SPRT_TOUR	FIDE chess, esports LAN fest	Chess, E-Sports	10

Students and advisors can export an Official Extracurricular and Activity Points Transcript at any time. It carries the college header and Dean's office, student metadata (name, roll number, branch, batch, semester), a cumulative summary with domain caps (for example Technical Innovation 40 of 40, Social and Community 20 of 30, Literary and Cultural 15 of 20, Sports and Mind Games 10 of 10, grand total 85 of 100 with the status 'On track for Degree Honors'), an itemised log with event, club, date, gate verification time, certificate identifier and points, and a QR code that points to the public verification URL. Because the ledger keeps every row even when a cap applies, caps can be revised later without losing history.

5.13 AI Copilot Suite

- **Hybrid event recommender:** combines content affinity between the student's interests and event tags, department and semester match, and past verified attendance. A simple explainable score is $S(u,e) = a \times \cos(I_u, G_e) + b \times D(u,e) + c \times H(u,e)$, where the weights a , b and c are non-negative and sum to one. Students with no history fall back to the first two terms.
- **Club Event Copilot:** from a short prompt, drafts a title, schedule, prerequisites, rules and promotional text. The output is only a draft that a Club Admin edits and the Dean reviews, so no AI text reaches students unreviewed.
- **Sentiment and suggestion clustering:** assigns a sentiment label and score to reviews and groups similar suggestions into topics, for example a Wi-Fi problem mentioned by many attendees becomes one actionable item.
- **Campus Concierge chatbot:** a retrieval-augmented assistant that answers in English and Hinglish about upcoming events, venues and club guidelines using platform data.

5.14 3D Campus Digital Twin

The campus twin renders low-poly models of the academic blocks, the auditorium, the innovation hub and the sports complex using Three.js and React Three Fiber. Buildings that host live or upcoming events display pulsing neon beacons. Selecting a building moves the camera and opens a drawer with venue details, events and registration links. Performance targets are more than 50 FPS on mobile and desktop with an asset bundle under 2 MB, achieved through instancing, simple materials and lazy loading.

CHAPTER 6

SECURITY ARCHITECTURE

6.1 Security Philosophy

CampusSphere enforces a zero-trust architecture. The frontend is never trusted for status, pricing or authorisation. Every state change is authenticated, authorised and recorded with an audit trail that cannot be repudiated. Cryptographic defences are applied against both physical and digital fraud: ticket screenshot forwarding, proxy check-in, forged certificates, fake reviews and payment tampering.

6.2 Token Lifecycle

On login the server verifies the password with BCrypt and issues a token pair. The access token uses HS512 or RS256, expires after 15 minutes (900 seconds) and carries the claims user identifier, role, roll number and administered clubs. The refresh token is a cryptographically random value stored in hashed form in the `refresh_tokens` collection and delivered only as an `HttpOnly`, `Secure`, `SameSite=Strict` cookie, so that cross-site scripting cannot read it. It is rotated at every refresh and revoked on logout or password change.

Table 6.1: Token specifications

Property	Access token	Refresh token
Lifetime	15 minutes	7 days
Storage on client	Memory (Authorization header)	<code>HttpOnly</code> <code>Secure</code> <code>SameSite=Strict</code> cookie
Server storage	None (stateless)	Hashed in MongoDB with rotation
Revocation	Expires quickly	Immediate on logout or password change
Content	sub, role, roll, clubs, exp, iat	Opaque random identifier

6.3 Anti-Screenshot Analysis

In a typical college event, a student registers, captures the QR pass as a screenshot and forwards it to friends. With a static code all copies are equally valid. With the rolling pass, the code is a function of the current 30-second window and a per-ticket secret, so a copy goes stale quickly and cannot be recomputed by anyone without the secret. Together with the single-use rule, this removes the economic value of sharing a pass.

Table 6.2: Fraud scenarios and defences

Scenario	Defence
Screenshot forwarded to a friend	Rolling HMAC token expires; single-use check at the gate
Two people scan the same ticket	Unique (event, user) attendance index; red alarm with first check-in time
Student lends a live phone	Not prevented cryptographically; scanner shows name and roll number for the volunteer to compare with the person
Fake review	Needs registration, COMPLETED event and gate-verified attendance
Edited certificate PDF	Seal mismatch on public verification
Tampered ticket price	Server computes price; client cannot specify amount
Fund diversion	Fees credited to the club ledger, read-only for the club

6.4 Payment Security and Split Accounting

The payment model has four layers. First, the server creates the order with a fixed server-calculated amount, so the client cannot choose the price. Second, the server verifies the HMAC signature of order and payment identifiers before issuing a ticket, and any mismatch is rejected and logged as a fraud attempt. Third, a unique index on the Razorpay payment identifier means duplicate webhooks or client replays are accepted idempotently without a second credit. Fourth, the club ledger is immutable for Club Admins: balances change only through backend transaction events, and each change is mirrored by an audit entry readable by the Super Admin.

6.5 Attendance-Gated Feedback Security

To protect organisers from smear campaigns and competitor spam, four rules apply: a confirmed registration must exist, the event must be COMPLETED, a gate-created attendance record must exist, and a unique (event, user) index allows one review per student. Any attempt to bypass these checks through API inspection returns 403 Forbidden and generates an audit notification.

6.6 Certificate Cryptography

Each certificate carries a public SHA-256 seal derived from its serial, roll number, event, issue date and a private college salt. Since the salt is known only to the college, an outsider cannot fabricate a record that verifies. The verification page reads values from the database, not from the PDF, so graphical edits of the PDF cannot influence the result. A limitation is that verification depends on the continued availability of the college's service; archiving verification records and publishing signed lists of seals would help alumni in the long term.

6.7 Threat Model (STRIDE)

Table 6.3: STRIDE threats and mitigations

Threat	Vulnerability	Mitigation
Spoofing	Student impersonates a Club Admin or HOD	Signed JWT with 512-bit HMAC secret; role claims checked on every endpoint
Tampering	Student changes ticket status from WAITLIST to CONFIRMED	Write-protected fields; status changed only by backend services
Repudiation	Admin denies modifying an event	Immutable audit logs with user, IP, time, action and diff
Information disclosure	Student reads club financial records	Authority checks on ledger reads; club scoping by token claims
Denial of service	Flash botting of registration endpoints	IP rate limiting via token bucket; edge DDoS protection; atomic DB updates
Elevation of privilege	Student grants self SUPER_ADMIN	Role field stripped from public registration and profile DTOs

6.8 Privacy and Data Protection

The platform stores names, roll numbers, institutional emails, interests, attendance and payment references, which are personal data under India's Digital Personal Data Protection Act, 2023. The design therefore collects only the fields needed for the stated purposes, stores no card or UPI credentials (these remain with the gateway), limits certificate verification output to what a recruiter needs, makes AI personalisation opt-in and defines retention for attendance and feedback data. Anonymous feedback keeps the verification flag but the author link is held separately so that clubs cannot identify reviewers.

6.9 Residual Risks

- A stolen refresh token on a compromised device is limited by rotation and revocation.
- A careless or malicious Club Admin could scan absent friends; controls are scanner identity in the audit log, comparison of the live counter with venue capacity and spot checks by the Dean.
- The 15-minute access token means a role revocation is not instant; a short deny-list for demoted admins is advisable, and club scope for financial operations should be re-validated against the database.
- The Dean's transcript URL is keyed by roll number, which is not secret; the public view should therefore show only a total and verification status.

CHAPTER 7

DEVELOPMENT PLAN, TESTING AND EVALUATION

7.1 Phased Development Roadmap

To produce production-quality software, the project follows ten phases. Each phase delivers a complete, self-contained slice and moves through the sequence: domain modelling, backend service and API, unit and integration tests, frontend UI, and end-to-end verification. A phase is closed only when its verification criteria pass. The trust-bearing modules (identity, clubs, events, registration, payments) come before attendance, credentials and AI, because the later modules depend on facts established earlier.

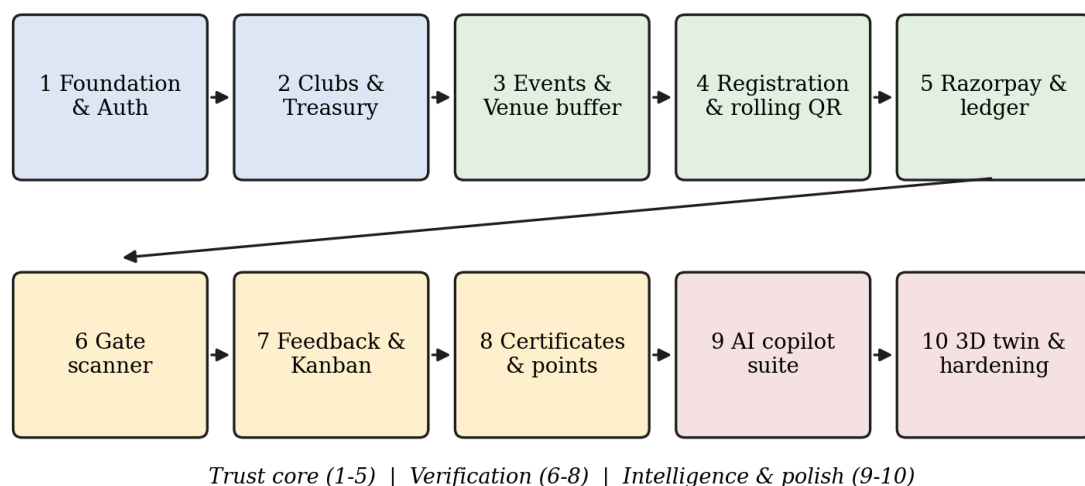


Figure 7.1: Ten-phase development roadmap

Table 7.1: Phases, deliverables and verification criteria

Phase	Deliverables	Verification criteria
1	Project structure, TypeScript models, MongoDB indexes, JWT with refresh rotation, RBAC guard, register/login UI	Registration with Arya roll number; login returns valid JWT; protected routes deny unauthorised access
2	Seeder for 15 clubs, club showcase, admin console, treasury models	All 15 clubs displayed; club admin can modify only own club
3	Venue catalog, event wizard, 30-minute buffer engine, approval queue	Overlaps rejected with alerts; DRAFT to PENDING_APPROVAL to APPROVED works
4	Atomic capacity lock, solo and team registration, rolling QR, My Tickets dashboard	Concurrent registrations never exceed capacity; expired QR tokens fail
5	Razorpay orders, signature verification, club ledger credit, Super Admin audit console	Payment verified server-side; balance and audit console update in real time

Phase	Deliverables	Verification criteria
6	Camera scanner PWA, token validation, single-scan guarantee, live counter	Valid scan approves; immediate second scan rejected with timestamp
7	Feedback gatekeeper, five-factor form, suggestion board, Kanban	Unverified students blocked; verified reviews carry the badge
8	PDF certificates, public verify portal, LinkedIn button, points engine, transcript	Certificate QR opens a valid public verification page
9	Recommender, event copilot, sentiment clustering, chatbot	Personalised scores; complete agendas; chatbot answers campus queries
10	3D digital twin, responsive and theme audits, Docker containers	Over 50 FPS; all phases integrate seamlessly

7.2 Testing Strategy

Testing is layered. **Unit tests** cover pure logic such as token derivation, window checks, buffer arithmetic and point calculation. **Integration tests** run the services against a test MongoDB replica set and verify transactions, unique indexes and idempotency. **API tests** exercise every endpoint against every role in an automated authorisation matrix. **Concurrency tests** fire many parallel requests at the capacity lock. **End-to-end tests** drive the browser through the main journeys. **Performance tests** check the latency targets, and a **mobile frame-rate profile** checks the 3D scene. Because the QR logic depends on time, the tests use an injectable clock so that boundary cases around 30 seconds are deterministic.

7.3 Representative Test Cases

Table 7.2: Acceptance test cases

ID	Scenario	Expected result
T1	500 parallel requests for an event with capacity 400	Exactly 400 confirmed; the rest told sold out
T2	Same payment webhook delivered 5 times	One payment row and one ledger credit
T3	Scan a token that is two windows old	Rejected as EXPIRED
T4	Scan the same ticket twice	Second scan returns 409 DUPLICATE_CHECK_IN with first time
T5	Book a venue 20 minutes after another event ends	409 VENUE_BUFFER_COLLISION with alternatives
T6	Book a venue 30 minutes after another event ends	Accepted
T7	Registered but not scanned student submits review	403 VERIFIED_ATTENDANCE_REQUIRED
T8	Student sends role=SUPER_ADMIN during signup	Role field ignored; account is STUDENT
T9	Edit name on PDF and open verification page	Seal mismatch; certificate invalid

ID	Scenario	Expected result
T10	Event completed with 120 verified attendees	120 ledger rows; each total increased by event points
T11	Club Admin of club A requests treasury of club B	403 Forbidden
T12	Tampered Razorpay signature	Rejected and logged as fraud attempt

7.4 Performance Targets and Measurement Plan

Table 7.3: Performance targets

Operation	Target	How it will be measured
Event discovery query	Under 200 ms for 10,000 events	Load test with indexed collections
Gate scan validation	Under 400 ms on 4G	p95 latency from scanner to response
Capacity reservation	Constant time, no overbooking	Parallel request test
3D campus scene	Over 50 FPS, under 2 MB	Frame profiler on mid-range phones
Venue conflict check	Logarithmic in bookings	Index explain plan

7.5 Risk Analysis

Table 7.4: Project risks and responses

Risk	Severity	Response
Weak campus Wi-Fi at the gate	High	Local validation with derived keys; manual roll-number entry with audit
Low adoption by clubs	High	Onboarding sessions; Dean mandate that club funds flow through ledgers
Payment gateway downtime during fest	Medium	Retry queue; free-event fallback; clear user messages
Ten-phase scope too large for the timeline	Medium	Vertical slices; ship phases 1 to 6 first
AI output quality or bias	Medium	Human approval of drafts; opt-out for personalisation
Personal data exposure	High	Data minimisation, encryption in transit, access logs

7.6 Observations on the Specification Set

A joint review of the specification documents found points that should be harmonised before coding begins, and the team recorded them for resolution:

- The rolling-token message is built from the registration identifier in the SRS but from the ticket number, user identifier and timestamp in the technical and security documents; one canonical format must be fixed.

- The replay bound is expressed both as a three-window check and as a 45-second limit; the stricter rule should be normative.
- An API example shows 24 seconds of validity while the pass is defined as 30 seconds.
- The stack is specified as either Spring Boot or Node.js, while some security text uses Spring annotation names and a role called ADMIN that does not exist among the three defined roles.
- Two groupings of activity points exist: eight event category codes and four ledger domains, with the transcript grouping Literary with Cultural and Social with Community. A single mapping table kept as data would let the Dean's office change caps without code changes.

7.7 Ethical Considerations

Because attendance and points influence a student's eligibility for graduation, errors have real consequences. A documented dispute path is therefore part of the design: a student who was present but not scanned can raise a request, the Club Admin and the faculty coordinator confirm with evidence, and the override is written to the audit log with both identities. Point crediting is never silent, so that errors become visible early and not in the final year.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

This project addressed a practical problem that every technical institution faces: how to run, fund, verify and credit student activities in a way that is transparent and trustworthy. The result of the work is CampusSphere, a complete design for an integrated platform for Arya College of Engineering & I.T., Jaipur. The design links the full chain of an extracurricular activity to facts that the platform can check: a server-priced payment credited to a club ledger, a rolling cryptographic pass scanned once at the gate, and a completed event that automatically yields the right to review, a sealed certificate and AICTE activity points.

From the engineering point of view, the design isolates the correctness-critical operations into steps that are atomic or idempotent: the conditional seat counter, the unique payment index, the single-scan attendance index and the transactional ledger credit. It rests on well-understood primitives, namely HMAC, SHA-256, JWT-based role control and MongoDB transactions, and it was checked against the STRIDE threat categories. The ten-phase roadmap with explicit verification criteria gives the team a controlled way to build and test the system.

The group gained practical understanding of requirement engineering in IEEE 830 style, document-oriented data modelling, REST API design, applied cryptography for fraud prevention, payment-gateway integration patterns, concurrency control, and the planning of a large software project in phases. Working as a team of five also strengthened skills in communication, review of each other's specifications and shared ownership of a common design.

8.2 Limitations

- This report documents analysis, design and the implementation plan; the performance and adoption targets are goals for the pilot and have not been measured yet.
- Certificate verification relies on a single institution-held salt and database, rather than an interoperable credential standard.
- Phone lending (a present student showing a live pass for someone else) can only be handled procedurally.

- Refund handling and bank settlement reconciliation for club payouts are tracked in the ledger model but not fully specified.
- The AI modules process student data and need privacy notices and opt-out controls before deployment.

8.3 Future Scope

- Implementing and piloting the phased roadmap with one or two clubs, then publishing measured latency, concurrency and fraud-detection results.
- Adopting standard verifiable-credential formats such as Open Badges or W3C Verifiable Credentials for certificates.
- Using per-event derived scanner keys for stronger offline validation.
- Native Android and iOS applications with push notifications and NFC-assisted entry.
- Integration with the RTU examination and student information systems for automatic transcript submission.
- Extending the platform to other colleges of the Arya Group and to inter-college events.
- Evaluating the AI recommender on real attendance logs and adding explainable recommendations.
- Automated bank settlement and reconciliation of club payouts with the college accounts office.

REFERENCES

- [1]AICTE, “Mandatory extracurricular activity points for B.Tech degree,” All India Council for Technical Education, New Delhi, and Rajasthan Technical University, Kota, degree honours ordinances.
- [2]CampusSphere Project Team, “Product Requirements Document (PRD), v1.0.0-PROD,” Arya College of Engineering & I.T., Jaipur, 2026.
- [3]CampusSphere Project Team, “Software Requirements Specification (SRS), IEEE Std 830 format, v1.0.0-PROD,” 2026.
- [4]CampusSphere Project Team, “Entity Relationship Diagram and Database Specification (ERD), v1.0.0-PROD,” 2026.
- [5]CampusSphere Project Team, “Technical Requirements Document (TRD), v1.0.0-PROD,” 2026.
- [6]CampusSphere Project Team, “REST API Specification, v1.0.0-PROD,” 2026.
- [7]CampusSphere Project Team, “Security Architecture and Fraud Prevention Specification, v1.0.0-PROD,” 2026.
- [8]CampusSphere Project Team, “AICTE / RTU Activity Points Specification, v1.0.0-PROD,” 2026.
- [9]CampusSphere Project Team, “Engineering Development Roadmap, v1.0.0-PROD,” 2026.
- [10]IEEE, “IEEE recommended practice for software requirements specifications,” IEEE Std 830-1998, 1998.
- [11]H. Krawczyk, M. Bellare and R. Canetti, “HMAC: Keyed-hashing for message authentication,” IETF RFC 2104, Feb. 1997.
- [12]D. M’Raihi, S. Machani, M. Pei and J. Rydell, “TOTP: Time-based one-time password algorithm,” IETF RFC 6238, May 2011.
- [13]M. Jones, J. Bradley and N. Sakimura, “JSON Web Token (JWT),” IETF RFC 7519, May 2015.
- [14]National Institute of Standards and Technology, “Secure Hash Standard (SHS),” FIPS PUB 180-4, Aug. 2015.
- [15]R. S. Sandhu, E. J. Coyne, H. L. Feinstein and C. E. Youman, “Role-based access control models,” IEEE Computer, vol. 29, no. 2, pp. 38–47, Feb. 1996.
- [16]N. Provos and D. Mazières, “A future-adaptable password scheme,” in Proc. USENIX Annual Technical Conf., Monterey, CA, USA, 1999, pp. 81–91.
- [17]R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, Univ. California, Irvine, CA, USA, 2000.

- [18]A. Shostack, Threat Modeling: Designing for Security. Indianapolis, IN, USA: Wiley, 2014.
- [19]M. Kleppmann, Designing Data-Intensive Applications. Sebastopol, CA, USA: O'Reilly Media, 2017.
- [20]MongoDB Inc., "Transactions," MongoDB Manual 7.0. [Online]. Available: <https://www.mongodb.com/docs/manual/>
- [21]Razorpay, "Payment gateway and webhook documentation." [Online]. Available: <https://razorpay.com/docs/>
- [22]Meta Platforms, "React documentation." [Online]. Available: <https://react.dev/>
- [23]Three.js Authors, "Three.js: JavaScript 3D library." [Online]. Available: <https://threejs.org/>
- [24]Government of India, "The Digital Personal Data Protection Act, 2023," Act No. 22 of 2023, Aug. 2023.